

UAVXGS — Ground Control Station for UAVXArm

From Windows-Only to Cross-Platform

UAVXGS is a complete reimplementaion of the original UAVXGUI ground control station. The legacy application was written in C# for the .NET Framework, tying it to Microsoft Windows and making field use inconvenient for macOS and Linux pilots. UAVXGS replaces it with a Python 3 / PyQt5 application that runs on any platform with a Python interpreter and a Qt5 binding.

Architecture

UAVXGS follows a modular structure under `uavx-python/src/`:

```
src/
  main.py                # Entry point
  protocol_constants.py  # Wire-format constants
  protocol_enums.py      # Packet tags, param indices, state enums
  packet_parser.py       # Binary-to-object deserialisation
  parameters.py          # Display multipliers, limits, types
  core/
    data_manager.py      # Flight-data state
    packet_logger.py     # Logging to CSV
    ack_handler.py       # Acknowledge processing
    speech.py            # Voice feedback (TTS)
  ui/
    main_window.py       # Primary instrument panel
    parameter_window.py  # Parameter read/write/save/load
    nav_window.py        # Mission planning / map
    calibration_window.py # IMU / mag calibration
    misc_window.py       # Raw-packet monitor
    dfu_flasher.py       # STM32 DFU firmware upload
  widgets/
    attitude_indicator.py # 3D attitude sphere with bars
    (other widgets)
  airframes/             # Param-set files (.af)
  models/
  missions/
  logger/
    logger.py            # CSV telemetry logger
    kml_logger.py        # KML / GPX track export
  tests/
  resources/
```

Entry Point

`main.py` creates a `QApplication`, instantiates `MainWindow`, opens the serial connection (or starts emulation), and enters the Qt event loop.

Serial Protocol Handling

Packet Framing

The FC transports frames over a SLIP-like protocol:

FF SOH [ESC-stuffed payload] checksum EOT CR LF

Stuffed bytes: SOH (0x01), EOT (0x04), ESC (0x1B). The escape prefix byte is excluded from the checksum. A dedicated serial tap (`serial_tap.py` or the compiled `intercept_serial.so`) performs ESC un-stuffing before packets reach the parser.

Deserialisation

`packet_parser.py` contains a `parse_packet()` dispatcher that uses `match/case` on the packet tag (byte 1 of the payload). Each tag maps to a specialised parser returning a typed dataclass:

Tag	Name	Data Class	Description
16	Control	<code>ControlData</code>	Desired throttle, angles, rates, drives
17	Param	<code>ParameterData</code>	Single-param write
19	Origin	<code>OriginData</code>	Home position
20	Waypoint	—	Mission waypoint
54	BBDump	—	Black-box chunked dump
57	Tuning	<code>TuningData</code>	Rate tracking + throttle + flags
59	Guidance	<code>GuidanceData</code>	GPS + velocity + heading
60	AltCtrl	<code>AltitudeControlData</code>	Altitude hold state
62	Calibration	<code>dict</code>	IMU calibration status
63	Config	<code>dict</code>	Firmware revision string
64	Wind	<code>WindData</code>	Wind estimation
66	SerialPorts	<code>dict</code>	Serial port configuration
67	ExecTime	<code>ExecTimeData</code>	CPU load
68	AttCtrl	<code>AttitudeControlData</code>	Per-axis attitude control (legacy)
78	Flight	<code>FlightData</code>	Attitude, altitude, battery, GPS

Protocol Enums

`protocol_enums.py` defines Python `IntEnum` classes mirroring the C-side enums:

- `PacketTag` — 128 values (0-127), one per tag
- `ParamIndex` — symbolic names for parameter indices
- `FlightState`, `NavState` — autopilot state machine
- `Config1Bits`, `Config2Bits` — airframe configuration bitfields
- `MiscCommand` — calibration and utility commands

This eliminates magic numbers and keeps the GCS code synchronised with the FC protocol definition.

Parameter System

Unified Float Handling

All 128 parameters are stored and transmitted as `float32` (IEEE-754). This is a deliberate departure from the legacy system that mixed float and `uint8` storage.

- **Display formula:** `display_value = fc_float × PARAM_DISPLAY_MULT[idx]`
- **Write formula:** `fc_float = display_value / PARAM_DISPLAY_MULT[idx]`
- **Range enforcement:** `PARAM_LIMITS[idx][0] × mult` to `PARAM_LIMITS[idx][1] × mult`
- **No legacy mode:** No `uint8` display conversion, no `LEGACY_TAGS`, no `PARAM_SCALES`

Airframe Files

`.af` files store complete parameter sets in JSON-like tagged format. All values are in FC-native units (volts, radians, metres). The airframe selector in `main_window.py` populates from `airframes/` and auto-selects the default on first launch.

UI Components

Main Window (`main_window.py`)

The primary instrument panel displays:

- **3D attitude sphere** (`AttitudeIndicator` widget) — pitch/roll with artificial horizon
- **Altimeter + VSI** — altitude tape and vertical speed trend
- **Speed tape** — groundspeed (GPS) and airspeed (if available)
- **Heading compass** — magnetic heading with cardinal markers
- **Throttle bar** — 0-100% with cruise-throttle reference line
- **Battery box** — voltage, current, consumed mAh
- **GPS status** — satellites, HDOP, fix type
- **Flight mode indicators** — arm, attitude mode, alt-hold, nav mode
- **Flags box** — configuration bitfield status
- **Status bar** — serial throughput, packet rate, link quality
- **Revision label** — firmware name from tag 63 (or fallback "UAVX r{N}")

Parameter Window (`parameter_window.py`)

Tabular parameter editor with:

- Grouped parameter categories (Basic, Gains, Rates, Altitude, Navigation, Misc, Tuning)
- Live read/write via tags 71/17
- File save/load for `.af` export
- Revision check on connect

Navigation Window (`nav_window.py`)

Map-based mission planner:

- OpenStreetMap tiles via `QWebEngineView`
- Waypoint drag-and-drop, right-click menu (VIA, ORBIT, PERCH, SETPOI, SURVEY)
- Mission upload (tag 19 + tag 20 writes) and download
- Real-time aircraft position overlay
- GoTo mode for one-click waypoint insertion

Calibration Window (`calibration_window.py`)

Guided calibration for accelerometer, gyroscope, and magnetometer, with live sensor readback and progress indication.

Misc Window (`misc_window.py`)

Raw-packet monitor displaying every received telemetry frame by tag. Useful for protocol debugging and bandwidth analysis.

DFU Flasher (`dfu_flasher.py`)

STM32 DFU mode firmware upload via `stm32_uart_bl.py`. No external programmer required — flashes over the existing UART connection.

Data Management

Data Manager (`core/data_manager.py`)

A singleton that holds the canonical state of all decoded telemetry data. Components read from it rather than parsing packets directly. This decouples UI rendering from packet handling and allows multiple windows to share the same data without duplication.

Packet Logger (`core/packet_logger.py`)

Logs every received packet to a rotating CSV file: timestamp, tag, direction (R/T), body bytes. This produces a canonical record for post-flight analysis and protocol debugging.

Telemetry Logger (`logger/logger.py`)

Writes a flight-log CSV at ~10 Hz containing: time, latitude, longitude, altitude, heading, speed, battery, throttle, attitude, rate setpoints, and rate feedback. Designed for import into MATLAB, Excel, or pandas for post-flight analysis.

Voice Feedback

`core/speech.py` implements configurable text-to-speech announcements using `espeak` (or any TTS engine). Events such as “armed”, “altitude hold engaged”, “RTH”, “low battery”, and “failsafe” can be spoken aloud, enabling the pilot to keep eyes on the aircraft.

Emulation Mode

When no serial port is available, the GCS starts in emulation mode. The FC (compiled with emulation support in `emu.h`) generates synthetic sensor data with a light breeze to the NE. Emulation mode is identical to live operation from the GCS perspective, making it a safe environment for mission-planning practice and parameter tuning.

Build and Deployment

UAVXGS requires:

- Python $\geq$ 3.10 (for match/case in packet_parser.py)
- PyQt5
- pyserial
- espeak (optional, for voice feedback)
- pandoc (optional, for PDF report generation)

No .NET runtime, no Windows DLLs, no Mono. The application launches from a terminal with:

```
cd uavx-python/src && python3 main.py
```

Key Differences from Legacy UAVXGUI

Aspect	Legacy UAVXGUI	UAVXGS
Language	C# (.NET Framework)	Python 3
UI toolkit	Windows Forms	Qt5 (PyQt5)
Platform	Windows only	Linux, macOS, Windows
Deployment	MSI installer	git clone && python3 main.py
Packet parsing	Manual binary read	Typed dataclass deserialisation
Map tiles	Bing Maps (API key required)	OpenStreetMap (free, offline-capable)
DFU flashing	External tool required	Built-in via UART
Parameter format	Mixed uint8/float	Unified float32
Parameter export	Proprietary binary	.af (readable tagged format)
Voice feedback	SAPI (Windows only)	espeak (all platforms)
Telemetry logging	None	CSV + KML export
Raw packet monitor	None	Built-in misc window